

AI-Assisted Development — Exam Submission

HotFixAmbulance — AI-Powered Production-Error Triage for .NET Web APIs

Author: Stanislav Stanev

AI tools used: Claude Code (primary)

Repository: <https://github.com/myPOSTech/mps-banking-hot-fix-ambulance>

Working-system evidence: <docs/evidence/working-system-evidence.pdf>

1. Project Overview

The idea

On-call engineers waste the first, most expensive minutes of an incident doing the same manual ritual: open Kibana, filter the last few hours of errors, eyeball the stack traces, guess the root cause, and go hunting through Git for the offending change. **HotFixAmbulance** automates that ritual. It is a triage tool that, for a chosen .NET Web API and time window, pulls the recent error logs, groups them into distinct problems, and — for each problem — uses a **local Qwen LLM** to write two human-readable columns: “**Suggestion for Error**” (what the error means) and “**How to fix**” (a concrete remediation), grounded in the service’s own Git history. The result is a ranked, explained, actionable error table in a web UI, plus a CLI for terminal/CI use.

System requirements

Functional

- Fetch Fatal/Error/Warning logs for a named API over a relative (last N h) or absolute time window from **Elasticsearch**.
- Group logs by exception fingerprint (exception type + normalized message + endpoint) and rank by severity, count, recency.
- For each group, produce an AI **Suggestion** and **How-to-fix**, using a **locally hosted Qwen model** and grounding the answer in Git blame + related commits.
- Persist every run; expose it through a REST API and a paginated React table; provide a CLI that triages and prints JSON/table output.
- The UI must make it **visible and verifiable** that a group was analysed by the LLM (a per-group 🤖 **Qwen** badge).

Non-functional

- **Graceful degradation:** the LLM is optional — if the model is unreachable, the system silently falls back to a deterministic Git-history heuristic and never fails a run.
- **Local & private:** the model runs on the developer’s machine in Docker (CPU-only), no data leaves the host.
- **Reproducible:** one command (`scripts/demo.ps1`) brings up the whole stack (Qwen, Elasticsearch, SQL Server), generates sample errors, runs triage, and opens the UI.

Tech stack

.NET `undefined` / C# `undefined` (backend, CLI) · ASP.NET Core Minimal APIs · Entity Framework Core + SQLite · Elasticsearch `undefined` · LibGit2Sharp · React `undefined` + TypeScript + Vite + TanStack Table + Tailwind · Ollama-runtime serving **Qwen 2.5 (3B)** · Docker Compose · PowerShell automation · xUnit + FluentAssertions + NSubstitute (backend) · Vitest + Testing Library (frontend).

2. System Architecture — Modules

The system was decomposed (with AI assistance) into focused technological modules, each a separate .NET project or front-end/infra unit with a single responsibility and a clear seam (interface) to its neighbours.

Testing strategy. Pure unit tests assert grouping keys, ranking order and correlation-id counting — fast and exhaustive because the layer is side-effect-free.

AI tool choice. Claude Code — strong at generating table-driven ranking tests and edge cases.

Prompts: “Group logs by a stable fingerprint and rank Fatal>Error>Warning, then count desc, then last-seen desc.” · “Add a normalizer that replaces GUIDs/numbers so the same error collapses into one group.”

3.3 AI Enrichment with Qwen (the centrepiece)

Approach & reasoning. The AI columns are owned by an `IGroupEnricher` seam so the strategy is swappable from config (`Analysis:Strategy`). `LlmGroupEnricher` builds a strict-JSON prompt (`{suggestion, howToFix}`) from the group + Git evidence, calls the model via `ILlmClient` (`OllamaLlmClient` , `temperature 0.2` , `format:"json"`), and on **any** failure (timeout, unreachable, bad JSON) degrades to the deterministic Git heuristic. A successful call tags the group `AnalyzedBy = "Llm"` ; that single fact drives the UI badge.

Step-by-step workflow. (1) `LlmPromptBuilder` (system contract + fact dump); (2) `OllamaLlmClient` that never throws (returns `null` on failure); (3) `LlmGroupEnricher` with fallback; (4) propagate per-group `AnalyzedBy` through `TriageService` → `ErrorGroup` → API → UI.

Testing strategy. Unit tests with a substituted `ILlmClient` assert: valid JSON → `Llm` ; null/garbage → heuristic fallback; and that `TriageService` stamps each group with the enricher’s source.

AI tool choice. Claude Code — it implemented the fallback contract, the run-level `Mixed/Llm/Heuristic` aggregation, and the tests in one TDD loop.

Prompts: “Add an LLM enricher that asks the model for {suggestion, howToFix} as JSON and falls back to the Git heuristic on any error — never throw.” · “Make sure every group records which strategy produced it, and surface a per-group marker in the UI proving Qwen was used.”

3.4 Git-History Grounding

Approach & reasoning. An LLM answer is only useful if grounded in the real code, so `FixHintBuilder` (behind `IFixHintSource`) uses `LibGit2Sharp` to fetch the offending file’s blame + a code snippet and search `origin/main` for related commits; this evidence is fed to both the heuristic and the Qwen prompt.

Step-by-step workflow. (1) `IGitRepoCache` to clone/pull per API; (2) `IGitHistoryReader` for blame + keyword commit search; (3) `FixHintBuilder` to assemble a compact evidence string.

Testing strategy. Unit tests substitute the repo cache/reader and assert the formatted hint contains the expected `file:line` , blame and commit SHAs.

AI tool choice. Claude Code — handled the `LibGit2Sharp` specifics and kept the evidence format testable.

Prompts: “Given a stack file+line, return blame + a code snippet from origin/main and the related commits.”

3.5 Persistence & REST API

Approach & reasoning. Runs are stored as a row with the groups serialised to JSON (SQLite), keeping the schema trivial and migration-free; the API is ASP.NET Core Minimal APIs with server-side paging/sorting (`GroupPager`).

Step-by-step workflow. (1) `HotFixDbContext` + `TriageRun` (incl. nullable `AnalyzedBy` for back-compat); (2) repository; (3) endpoints: run triage, latest, by-id, paged groups, history.

Testing strategy. Integration tests with `WebApplicationFactory` hit the real endpoints (in-memory/SQLite), asserting status codes, paging bounds and the `analyzedBy` field round-trips.

AI tool choice. Claude Code — generated endpoints + integration tests and kept DTOs in sync with the frontend types.

Prompts: “Expose paginated groups with sort/dir validation and an allowed page-size list, plus a run-by-id endpoint.”

3.6 Frontend UI + the Qwen badge

Approach & reasoning. A `TanStack-Table` grid with toggleable columns; the AI columns render a small violet

 Qwen badge iff `analyzedBy === 'Llm'` — the visible proof the model was used.

Step-by-step workflow. (1) typed API client + `ErrorGroup` type incl. `analyzedBy`; (2) `QwenBadge` component; (3) render it in the Suggestion and How-to-fix cells; (4) metrics panel (“AI Insights Generated”).

Testing strategy. Vitest + Testing Library: badge appears for `Llm`, is absent for `Heuristic` / `null` — written **before** the component (TDD red→green).

AI tool choice. Claude Code — wrote the failing tests first, then the component, matching the repo’s TDD rule.

Prompts: “Render a Qwen badge on the AI columns only when analyzedBy is 'Llm'; add Vitest tests for all three cases first.”

3.7 Infrastructure & DevOps

Approach & reasoning. Mirror the existing `infra/<service>` pattern: a CPU-only Qwen runtime in Docker Compose (`infra/qwen`), an idempotent `bootstrap.ps1` that pre-pulls `qwen2.5:3b`, and a `demo.ps1` that orchestrates the whole stack and **asserts** the produced run was analysed by the LLM before opening the UI.

Step-by-step workflow. (1) compose + healthcheck; (2) bootstrap (pull + chat probe); (3) corporate-CA injection (`export-host-ca.ps1` + `docker-compose.corp.yml`) for TLS-intercepting proxies; (4) wire `-WithLlm` /default LLM env into `demo.ps1`.

Testing strategy. Each script is verified live: compose `config`, container health, `/api/tags`, model present, idempotent re-run, and an end-to-end demo asserting `analyzedBy ∈ {Llm, Mixed}`.

AI tool choice. Claude Code — it ran the Docker/PowerShell commands, diagnosed failures from their output, and iterated.

Prompts: “Make Qwen part of the infrastructure in Docker like Elasticsearch, and guarantee the model is present before the app starts.” · “The container can’t verify TLS through our proxy — inject the corporate CA.”

3.8 Testing & Quality (cross-cutting)

Approach & reasoning. A mandatory TDD cycle (failing test first) and a **pre-commit hook** that blocks any commit unless `dotnet test`, `dotnet format`, frontend tests and lint all pass — so `main` /integration stays green.

Step-by-step workflow. Write the failing test → minimal implementation → run → commit (hook gates it).

Testing strategy. `undefined` backend unit tests + `undefined` integration tests + `undefined` frontend tests, all green at each commit.

AI tool choice. Claude Code — drove the red→green→commit loop and respected the project’s skills/hooks.

Prompts: “Add the failing test for AnalyzedBy first, watch it fail, then implement and run the suite.”

4. Challenges & Tool Comparison

Biggest challenges

- **Corporate TLS interception.** The Docker container could not `ollama pull` the model — `x509: certificate signed by unknown authority` — because the corporate proxy MITM-terminates TLS and the container lacked the corporate root CA. **Solved** by exporting the host CA bundle (`export-host-ca.ps1`) and mounting it via a `docker-compose.corp.yml` overlay with `SSL_CERT_FILE`. This was the single hardest, most environment-specific problem.
- **A latent DI bug surfaced by the new feature.** The CLI had never been wired to the `IGroupEnricher` seam added in a prior phase, so it crashed with “Unable to resolve service for type IGroupEnricher.” Found it by running the CLI; fixed by registering `AddHotFixGroupEnrichment` — which also enabled the CLI to use Qwen.
- **CPU inference latency.** One LLM call per group, sequential, on CPU (~10 s each) makes a 38-group run take minutes; mitigated by warming the model and raising the API client timeout. A real trade-off of running a private model locally.
- **Tooling papercuts.** PowerShell 5.1 mis-parsing a UTF-8 script because of em-dash characters (replaced with ASCII); Chrome headless needing absolute paths and an origin before `localStorage`; SQLite file-locks when a running API blocked the build. Each was diagnosed from command output and fixed.

Which tool helped the most, and why

Claude Code was the primary and most valuable tool. Beyond code generation it could **run** the project — execute Docker/PowerShell/dotnet, read the failures, and iterate — which is what actually solved the infrastructure problems (the CA injection, the DI crash, the parse bug). Its skills/sub-agents kept the work disciplined (plan → TDD → verify → commit with a passing pre-commit gate), and it could reason across the whole repo (backend + frontend + infra) to keep types, DTOs and the demo orchestration consistent end-to-end.

What I would improve next

- **Batch / parallelise** the per-group LLM calls (or stream) to cut run time; optionally cache by fingerprint.
- Add a **GPU compose overlay** for much faster inference where a GPU is available.
- Make the persisted strategy tag itself read **Qwen** (currently the internal tag stays the generic **Llm**, while the UI shows “Qwen”).
- Share one SQLite store between CLI and API so a CLI-produced run is viewable in the UI without a second pass.
- Add a small **eval harness** that scores the model’s suggestions against known root causes.

5. Working System Evidence

See [docs/evidence/working-system-evidence.pdf](#) (4 captioned figures). Highlights:

- **Figure undefined — Overview:** a real run for **demo-api** — **73 logs** → **undefined groups**, with **38 “AI Insights Generated”** and **38 “Fix Recommendations.”**
- **Figure undefined — Proof of Qwen:** every AI cell shows the  **Qwen** badge and genuine, context-specific model text

(e.g. “...failed database save operation at line **undefined** in DemoDatabase.cs...”, “...invalid SQL object name 'Pricing.Records'...”).

- **Figure undefined — Full table:** all **38 groups** carry the badge (run-level **analyzedBy = "Llm"**).
- **Figure undefined — Terminal:** **docker ps** (Qwen healthy on :11434), **ollama list** (**qwen2.5:3b** , 1.9 GB), API run **analyzedBy = Llm** with **withSuggestions = 38** / **withFixes = 38** , and the container at **1177 % CPU** during inference.

Reproduce locally:

```
powershell -File scripts/demo.ps1 -WithElastic -KeepRunning
# then open the printed UI URL – every analysed group shows the  Qwen badge
```

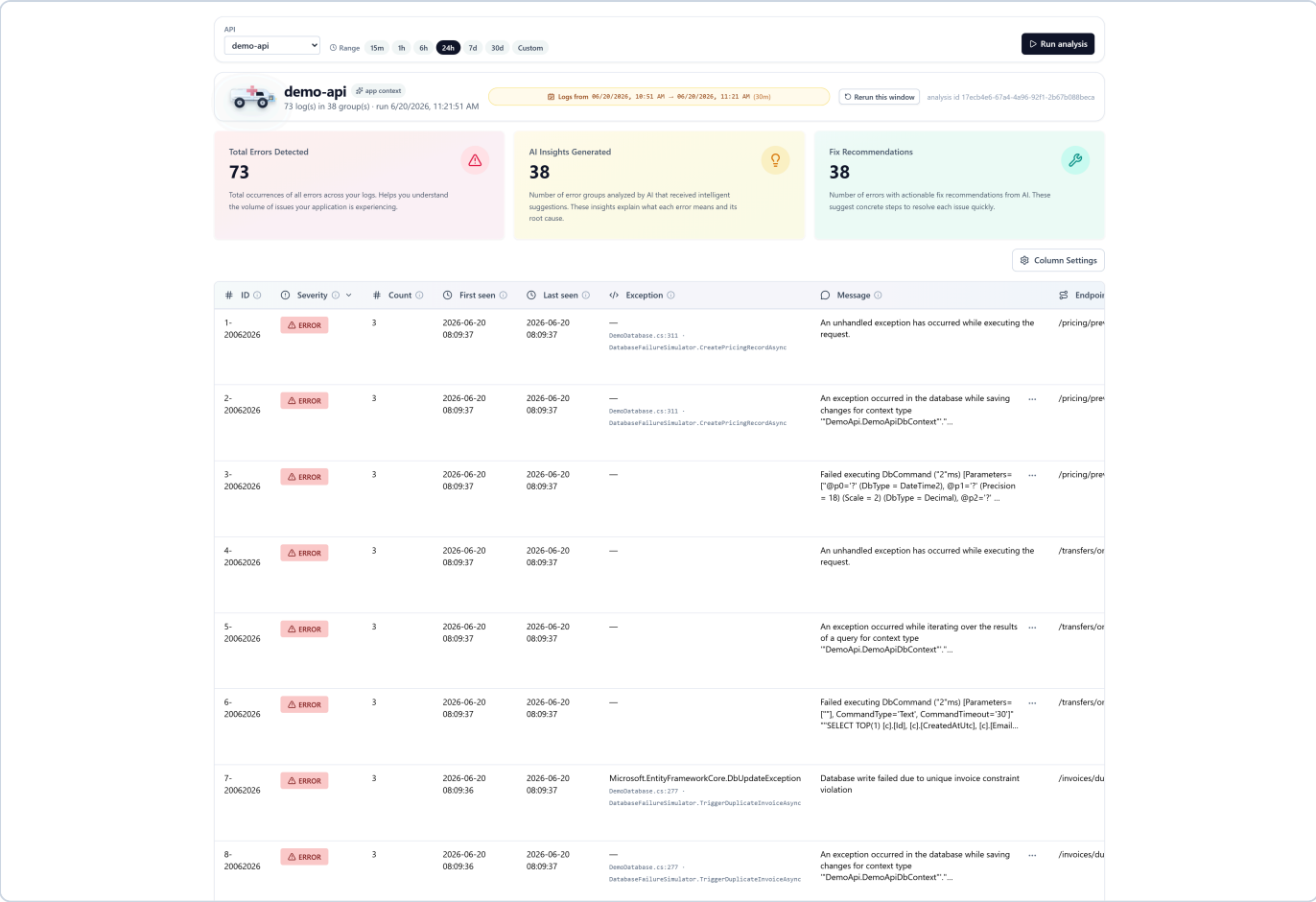


Figure 1 — System overview (ingestion, grouping, AI layer ran). A real triage run for the **demo-api** service: **73 error logs** collapsed into **38 distinct groups**. The metrics panel confirms the AI layer ran end-to-end — **38 "AI Insights Generated"** and **38 "Fix Recommendations"** (every group enriched). The table shows each group's severity, count, first/last-seen, exception with stack frame, message and endpoint — the raw evidence read from Elasticsearch. *Proves the ingestion → grouping → analysis pipeline works.*

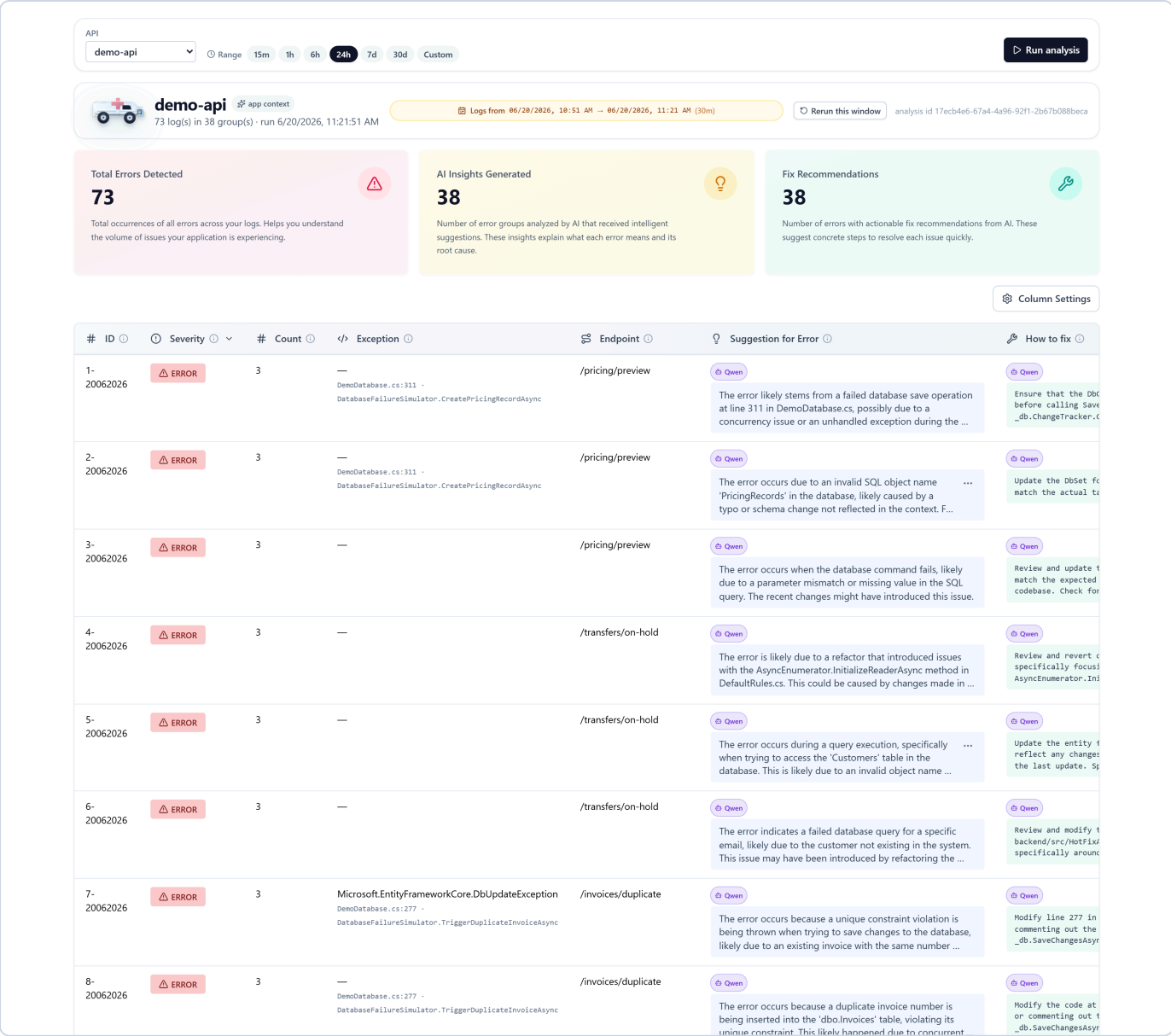


Figure 2 — Proof the analysis is produced by Qwen. The two right-most columns are the AI columns; each cell carries a violet  Qwen badge, rendered *only* when a group's analyzedBy ≡ 'Llm' — a value the backend sets exclusively after a successful Qwen completion. The text is genuinely model-generated and context-specific (e.g. "...failed database save operation at line 311 in DemoDatabase.cs...", "...invalid SQL object name 'Pricing.Records'..."), not a static template. *Proves the system demonstrably uses Qwen.*


```
# Qwen is served from Docker (CPU-only) on :11434
$ docker ps --filter name=hfa-qwen
hfa-qwen    ollama/ollama:latest    Up 59 minutes (healthy)    0.0.0.0:11434→11434/tcp

# qwen2.5:3b is pulled into the persistent volume
$ docker exec hfa-qwen ollama list
NAME          ID          SIZE      MODIFIED
qwen2.5:3b    357c53fb659c  1.9 GB    37 minutes ago

# A triage run produced via the API - analysed by the LLM, every group enriched
$ curl -s http://localhost:5283/api/triage/runs/17ecb4e6-... /
apiName      = demo-api
analyzedBy   = Llm
totalLogs    = 73
totalGroups  = 38
summary.withSuggestions = 38    # all 38 groups got an AI suggestion
summary.withFixes      = 38    # all 38 groups got an AI fix

# During inference the Qwen container saturates the CPU (one call per group)
$ docker stats hfa-qwen --no-stream
hfa-qwen    CPU = 1177%    MEM = 4.4 GiB / 15.4 GiB
```

Proves Qwen runs locally in Docker, the model is loaded, and a real run returns `analyzedBy = Llm` with an AI suggestion + fix for every one of the 38 groups.

6. Repository

GitHub: <https://github.com/myPOSTech/mps-banking-hot-fix-ambulance>

(Feature work on branch `integration-llm`.)